

Hồ sơ bàn giao tối ưu cho Technical Assessment Ticket Booking Backend

Tóm tắt điều hành

Đọc sát đề bài cho thấy reviewer **không yêu cầu một hệ thống production-ready**, mà muốn đánh giá ba thứ: cách bạn tư duy về backend design, cách tổ chức codebase, và cách reasoning về các issue/workflow có thể xảy ra. Đề cũng yêu cầu toàn bộ kết quả được gom vào một folder/link để submit. Ở phần deliverables, đề nêu trực tiếp: tài liệu System/Database Design có phân tích và giải thích, source code, coding guideline/convention, hướng dẫn setup local, API documentation như Swagger, API testing collection như Postman chạy được với local setup, và tài liệu assumptions / implemented scope / not implemented / limitations. filecite~~turn0~~file0~~file~~

Vì vậy, bộ hồ sơ tốt nhất không nên là một repo có rất nhiều Markdown rời rạc. Nó nên kể được một câu chuyện liên tục:

```
Business problem
  ↓
Assumptions + invariants
  ↓
Architecture decision
  ↓
Database guarantees
  ↓
Booking transaction
  ↓
API contract
  ↓
Executable implementation
  ↓
Automated tests
  ↓
Concurrency/load evidence
  ↓
Known limitations + future evolution
```

Với assessment này, tôi đề xuất chia deliverables thành ba lớp:

Mức	Ý nghĩa	Nội dung
Explicitly required	Đề bài yêu cầu trực tiếp	System/DB Design, source code, coding guideline, local setup, API docs, API collection, assumptions/scope/limitations

Mức	Ý nghĩa	Nội dung
Treat as mandatory	Không phải mọi mục đều được gọi tên trong PDF nhưng thiếu chúng repo sẽ khó review hoặc khó reproduce	README, migration, seed, automated tests, Docker Compose, ADR
WOW / plus point	Bằng chứng engineering maturity	concurrency report, k6 flash-sale test, expiry-worker design, transactional outbox, observability, audit log, CI, benchmark/result artifacts

Đây là cách phân loại quan trọng: **đừng biến optional infrastructure thành mandatory complexity**. Assignment chỉ nêu khoảng 300–500 booking requests/phút nhưng đặc biệt lo ngại overselling, duplicate retry, voucher abuse và flash-sale instability. Vì vậy giá trị của submission nằm nhiều hơn ở **correctness evidence** hơn là số lượng technology. 📄filecite📄turn0file0📄

Bộ hồ sơ tôi khuyên nộp:

```

README.md                                ← reviewer entry point

docs/
├── EXECUTIVE_SUMMARY.md                  ← 1-page overview
├── SYSTEM_DESIGN.md                     ← architecture + workflows
├── DATABASE_DESIGN.md                   ← ERD + invariants + SQL guarantees
├── API_DESIGN.md                        ← optional companion to OpenAPI
├── ASSUMPTIONS_AND_LIMITATIONS.md       ← scope contract
├── TEST_STRATEGY.md                     ← test rationale
├── PERFORMANCE_REPORT.md                ← concurrency/k6 evidence
├── OBSERVABILITY.md                     ← optional plus point
├──
├── diagrams/
│   ├── architecture.mmd
│   ├── erd.mmd
│   ├── booking-sequence.mmd
│   ├── booking-state.mmd
│   └── reservation-timeline.mmd
├──
└── adr/
    ├── 001-modular-monolith.md
    ├── 002-postgresql-as-source-of-truth.md
    ├── 003-atomic-inventory-reservation.md
    └── 004-idempotency-strategy.md

openapi/
└── openapi.yaml

postman/
├── Ticket-Booking.postman_collection.json
└── Local.postman_environment.json

```

```

database/
├─ migrations/
└─ seeds/

tests/
├─ unit/
├─ integration/
├─ concurrency/
└─ load/
    └─ flash-sale.k6.js

test-results/
├─ concurrency/
├─ load/
└─ README.md

.github/
└─ workflows/
    └─ ci.yml

compose.yaml
Dockerfile
.env.example

```

OpenAPI là lựa chọn phù hợp để biến API contract thành artifact machine-readable: OAS hiện định nghĩa một interface description độc lập ngôn ngữ cho HTTP API và có thể được dùng bởi documentation, testing và code-generation tools. Phiên bản chính thức hiện hành của specification là OpenAPI 3.2.0; tuy nhiên nếu framework Swagger của bạn hỗ trợ 3.0/3.1 tốt hơn thì **tool compatibility quan trọng hơn việc cố dùng specification mới nhất**. ¹

Đối với visual documentation, Mermaid là lựa chọn rất hợp repository vì ERD, sequence diagram và state diagram đều có syntax chính thức và có thể version-control cùng code. Mermaid cũng có `timeline`, nhưng tài liệu hiện ghi timeline vẫn là experimental diagram; vì vậy timeline chỉ nên là asset phụ, còn booking race nên ưu tiên sequence diagram nếu bạn muốn stability tốt hơn. ²

Nguyên tắc cuối cùng: một reviewer nên clone repo, đọc README trong vài phút, chạy một command để dựng môi trường, gọi được booking API, chạy test và nhìn thấy bằng chứng rằng inventory không oversell.

Bản đồ deliverables và cấu trúc submission

Trang cuối của đề bài là nguồn sự thật chính để phân biệt requirement với enhancement. Nó yêu cầu System/Database Design có analysis/explanation; codebase gồm source, coding guideline, local setup, API docs, API testing collection; đồng thời phải mô tả assumptions, phần đã làm/chưa làm và limitations. Đề còn chủ động cho phép bạn thu hẹp feature như seed voucher thay vì làm full voucher CRUD, miễn là scope được định nghĩa rõ. [filecite](#)[turn0file0](#)

Ma trận tài liệu cần nộp

Artifact	Mức	Mục đích	Reviewer phải tìm được ngay
<code>README.md</code>	Mandatory-for-quality	Entry point của repo	Quick start, architecture snapshot, commands, docs map
<code>EXECUTIVE_SUMMARY.md</code>	Recommended	Cho reviewer hiểu solution trong 1 trang	Problem, decisions, correctness, trade-offs, demo
<code>SYSTEM_DESIGN.md</code>	Explicit requirement	Giải thích architecture và workflow	Architecture diagram, invariants, sequence, state machine, scaling
<code>DATABASE_DESIGN.md</code>	Explicit requirement	Chứng minh data consistency	ERD, schema rationale, constraints, indexes, transactions
<code>ASSUMPTIONS_AND_LIMITATIONS.md</code>	Explicit requirement	Chốt scope	Assumptions, done/not done, limitations, future
Source code	Explicit requirement	Implementation	Module boundaries rõ, conventions nhất quán
Coding guideline	Explicit requirement	Cho thấy maintainability	Add API workflow, naming, errors, transactions, tests
Migration	Mandatory-for-quality	Reproduce schema	Versioned migrations chạy từ DB rỗng
Seed	Mandatory-for-quality	Reviewer test nhanh	Concert, categories, inventory, voucher, users

Artifact	Mức	Mục đích	Reviewer phải tìm được ngay
OpenAPI / Swagger	Explicit requirement	API contract	Request/ response/ errors/header/ examples
Postman collection	Explicit requirement	Executable manual API test	Chạy trực tiếp với local env
Unit/integration tests	Mandatory-for-quality	Correctness evidence	Core business rules và DB behavior
ADRs	Mandatory-for-quality	Document trade-offs	Context/ options/ decision/ consequences
Docker Compose	Recommended mạnh	One-command local setup	API + PostgreSQL, optional Redis
Concurrency report	WOW	Chứng minh không oversell	N requests vs stock, exact assertions
k6 scripts/report	WOW	Chứng minh flash-sale behavior	Scenario, thresholds, p95/p99, errors
Outbox design	Optional WOW	Show future async reliability	Outbox table + publisher flow
Observability doc	Optional WOW	Production thinking	Logs, metrics, traces, correlation ID
Audit log	Optional WOW	Operation accountability	Manual booking-status changes
CI workflow	Recommended WOW	Reproducible quality gate	lint → test → integration → build

Docker Compose đặc biệt hữu ích cho assessment vì Compose cho phép định nghĩa multi-container application trong một YAML và khởi chạy stack bằng một command; tài liệu Docker cũng nêu Compose hữu ích cho isolated automated-testing environments. ³

Layout đề xuất

```
ticket-booking-assessment/
|
├── README.md
├── Dockerfile
├── compose.yaml
├── .env.example
|
├── docs/
│   ├── EXECUTIVE_SUMMARY.md
│   ├── SYSTEM_DESIGN.md
│   ├── DATABASE_DESIGN.md
│   ├── API_DESIGN.md
│   ├── ASSUMPTIONS_AND_LIMITATIONS.md
│   ├── TEST_STRATEGY.md
│   ├── PERFORMANCE_REPORT.md
│   └── OBSERVABILITY.md
|
│   ├── diagrams/
│   │   ├── architecture.mmd
│   │   ├── erd.mmd
│   │   ├── booking-sequence.mmd
│   │   ├── booking-state.mmd
│   │   └── reservation-timeline.mmd
|
│   └── adr/
│       ├── 001-use-modular-monolith.md
│       ├── 002-use-postgresql.md
│       ├── 003-atomic-inventory-update.md
│       └── 004-idempotent-booking-api.md
|
├── openapi/
│   └── openapi.yaml
|
├── postman/
│   ├── Ticket-Booking.postman_collection.json
│   └── Local.postman_environment.json
|
├── database/
│   ├── migrations/
│   └── seeds/
|
├── src/
│   ├── concert/
│   ├── booking/
│   ├── inventory/
│   ├── voucher/
│   ├── operation/
│   └── shared/
```

```

├── tests/
│   ├── unit/
│   ├── integration/
│   ├── concurrency/
│   └── load/
│       └── flash-sale.k6.js
├── test-results/
│   ├── concurrency/
│   └── load/
├── .github/
│   └── workflows/
│       └── ci.yml

```

Reviewer quick-check cho package tổng thể

Check	Pass khi
Entry point	Root có <code>README.md</code>
Reproducibility	Từ clone → running API bằng commands copy/paste được
Database	Migration + seed chạy trên DB sạch
Documentation	README link tới tất cả design docs
Visuals	Diagram source nằm trong repo, không chỉ screenshot
API	Swagger/OpenAPI mở được và Postman gọi local được
Testing	Có command riêng cho unit/integration/concurrency
Scope	Reviewer thấy ngay done/not done/limitations
Evidence	Không chỉ nói “safe under concurrency”; có test result
Traceability	ADR giải thích các architecture choice chính

Template one-page executive summary

`docs/EXECUTIVE_SUMMARY.md` nên ngắn đến mức reviewer đọc một lần là biết toàn bộ solution:

Executive Summary

Problem

This project implements the backend of a concert ticket booking platform supporting customer booking flows and internal operation workflows.

The main engineering risks addressed are:

- ticket overselling
- duplicate booking caused by retries

- voucher quota abuse
- temporary reservations
- flash-sale contention

Expected traffic from the assessment:

- ~50,000 users
- ~300-500 booking requests/minute

Scope

Implemented:

- concert and ticket-category query
- ticket availability
- booking reservation
- voucher application
- idempotent booking creation
- booking lifecycle
- reservation expiration
- operator booking inspection/status updates

Not implemented:

- real payment gateway
- seat-level assignment
- refund workflow
- notification delivery
- multi-region deployment

Architecture

Modular monolith + PostgreSQL.

PostgreSQL is the consistency boundary for:

- inventory
- booking
- voucher quota
- idempotency

Critical booking mutations execute in a single database transaction.

[Architecture diagram]

Correctness Guarantees

1. Inventory never becomes negative.
2. One idempotency key creates at most one logical booking.
3. Voucher quota cannot exceed its configured limit.
4. Failed booking transactions do not consume inventory/vouchers.
5. Expired reservations release reserved resources.
6. Invalid booking-state transitions are rejected.

Key Decisions


```
| Decision | Rationale |
|---|---|
| Modular monolith | Simple deployment and ACID transaction boundary |
| PostgreSQL | Strong consistency around finite resources |
| Conditional UPDATE | Prevent overselling without read-then-write race |
| Idempotency key | Retry-safe POST /bookings |
| Background expiry worker | Release abandoned reservations |
```

Verification

Concurrency scenario:

- Initial inventory: ____
- Concurrent attempts: ____
- Successful bookings: ____
- Rejected bookings: ____
- Final inventory: ____
- Overselling detected: YES / NO

Load test:

- RPS: ____
- p95 latency: ____
- p99 latency: ____
- unexpected error rate: ____

Run

`docker compose up --build`

See README.md for:

- migrations and seed
- tests
- Swagger
- Postman
- concurrency/load tests

Main Trade-offs / Limitations

[3-5 concise items]

Documentation

- System Design
- Database Design
- ADRs
- Assumptions & Limitations
- Test / Performance Report

Không điền số benchmark giả. Chỉ đưa con số đã chạy thật, kèm commit hash, environment và timestamp.

Tài liệu thiết kế bắt buộc

SYSTEM_DESIGN.md

Purpose: chứng minh bạn hiểu bài toán trước khi nói về framework. Đây là tài liệu quan trọng nhất cùng với source code vì để nói rõ reviewer quan tâm engineering thinking và trade-offs hơn số lượng feature. filecite turn0file

Minimum content nên theo thứ tự này:

Section	Nội dung bắt buộc
Context	Customer + Operation workflows
Goals	Reserve ticket, voucher, tracking, operations
Non-goals	Payment thật, seat selection, multi-region...
Traffic assumptions	50k users, 300-500 booking req/min
Critical risks	Oversell, retry duplicate, voucher abuse, flash spike
Business invariants	Các condition không bao giờ được phá
Architecture	Components/modules + dependencies
Booking workflow	End-to-end sequence
Transaction boundary	Những write nào atomic cùng nhau
Concurrency strategy	Atomic update / locks / ordering
Idempotency	Key, request fingerprint, replay/conflict
Booking lifecycle	State machine
Reservation expiry	Worker + race handling
Operations	Manual transition + audit
Scaling	Stateless API, DB contention, optional cache
Failure scenarios	DB rollback, duplicate requests, worker race
Trade-offs	Why not Redis lock / Kafka / microservices
Future evolution	Outbox, queue, horizontal scaling

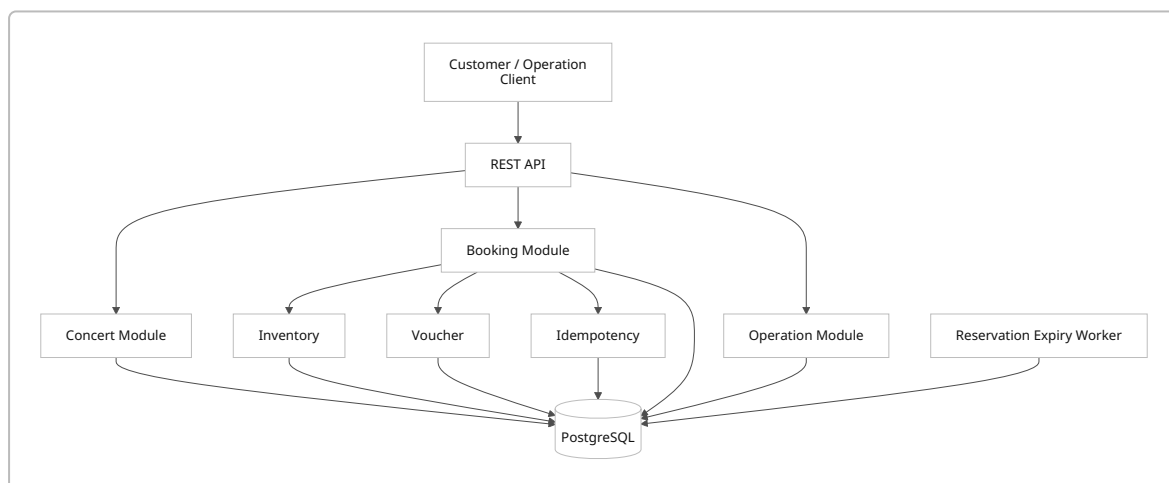
Business-invariant table template:

ID	Invariant	Enforcement	Test evidence
INV-01	Inventory cannot be negative	Conditional SQL + CHECK	CONC-001
INV-02	Same logical retry creates one booking	Unique idempotency constraint	CONC-002

ID	Invariant	Enforcement	Test evidence
INV-03	Voucher global quota not exceeded	Conditional SQL	CONC-003
INV-04	Booking creation is atomic	DB transaction	INT-004
INV-05	Expired reservation releases stock	expiry worker transaction	INT-005
INV-06	Invalid state transitions rejected	domain policy + conditional update	UNIT-006
INV-07	Historical price does not mutate	booking-item price snapshot	INT-007

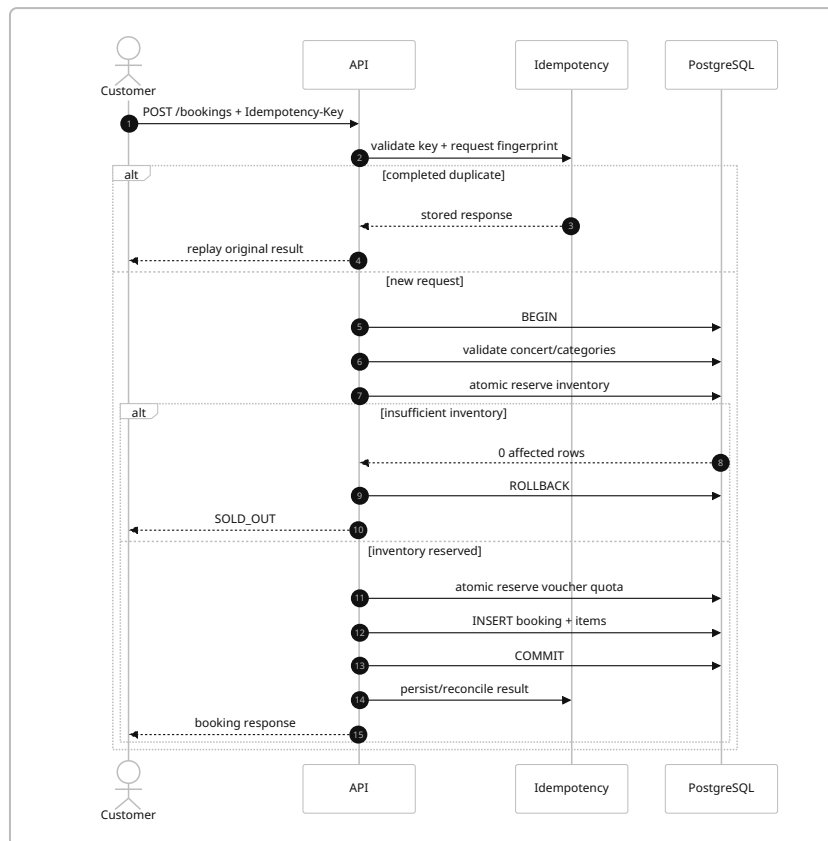
Tôi đặc biệt khuyến nghị bảng này vì nó liên kết **design** → **DB guarantee** → **automated proof**. Đó là kiểu traceability reviewer có thể verify thay vì phải tin prose.

Architecture visual cần có:



Mermaid hỗ trợ version-controlled diagrams trong Markdown, và sequence/state/ER diagrams đều có native syntax phù hợp trực tiếp với các asset của assessment này. 4

Booking sequence phải là asset bắt buộc:



Sequence diagrams biểu diễn thứ tự interaction giữa processes, và Mermaid hỗ trợ `alt`, `opt`, `par`, activation và automatic sequence numbers, nên rất phù hợp để ghi rõ cả happy path lẫn race/error path. 5

Reviewer quick-check — System Design

- [] Tìm thấy invariants trong dưới 1 phút.
- [] Biết transaction bắt đầu/kết thúc ở đâu.
- [] Biết chính xác cơ chế chống oversell.
- [] Biết retry hoạt động thế nào.
- [] Biết voucher concurrency xử lý thế nào.
- [] Có state machine và expiry behavior.
- [] Có failure cases, không chỉ happy path.
- [] Có trade-off rationale, không chỉ technology list.
- [] Architecture hiện tại khác future architecture rõ ràng.

DATABASE_DESIGN.md

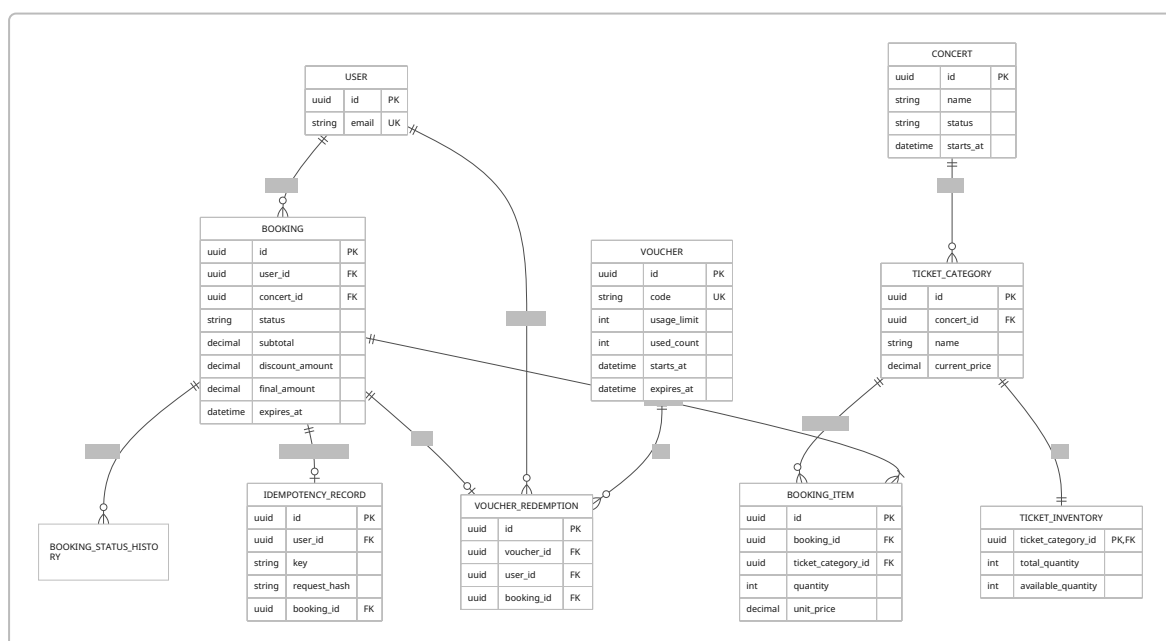
Purpose: chứng minh correctness được đặt xuống database thay vì chỉ phụ thuộc vào `if` trong application.

PostgreSQL hiện hỗ trợ `CHECK`, `UNIQUE`, primary/foreign-key constraints; unique constraint trên nhiều columns bảo đảm tính duy nhất của tổ hợp giá trị và PostgreSQL tự tạo unique B-tree index cho unique constraint. Đây là nền tảng phù hợp cho idempotency uniqueness và per-user voucher redemption. 6

Tài liệu DB nên chứa:

Section	Nội dung
Data model overview	Vì sao mỗi table tồn tại
ERD	Entity + cardinality
Table dictionary	Column/type/nullability/meaning
Constraints	PK/FK/UNIQUE/CHECK
Indexes	Query → index mapping
Transaction patterns	inventory/voucher/booking
Price snapshots	Historical consistency
Idempotency storage	scope/key/hash/result
Expiry queries	indexes + worker
Audit model	status history
Optional outbox	async evolution
Migration strategy	migration ordering/rollback assumption
Seed model	deterministic reviewer data

ERD asset:



Mermaid ER diagrams hỗ trợ cardinality, attributes và key markers như **PK**, **FK**, **UK**, đủ để version-control logical ERD ngay trong repo. 7

Table dictionary template:

Table.Column	Type	Null?	Constraint/ index	Business meaning
<code>ticket_inventories.available_quantity</code>	integer	No	CHECK	Quantity still reservable
<code>bookings.status</code>	enum/text	No	index	Current lifecycle state
<code>bookings.expires_at</code>	timestampz	Conditional	(<code>status</code> , <code>expires_at</code>)	Reservation deadline
<code>booking_items.unit_price</code>	numeric	No	—	Price snapshot
<code>vouchers.used_count</code>	integer	No	CHECK	Reserved/ used global quota
<code>idempotency_records.key</code>	text	No	UNIQUE composite	Retry identity

Constraint SQL nên đưa trực tiếp vào document:

```

ALTER TABLE ticket_inventories
ADD CONSTRAINT ck_inventory_non_negative
CHECK (available_quantity >= 0);

ALTER TABLE ticket_inventories
ADD CONSTRAINT ck_inventory_not_above_total
CHECK (available_quantity <= total_quantity);

ALTER TABLE idempotency_records
ADD CONSTRAINT uq_idempotency_user_key
UNIQUE (user_id, idempotency_key);

ALTER TABLE voucher_redemptions
ADD CONSTRAINT uq_voucher_redemption_user
UNIQUE (voucher_id, user_id);

```

`CHECK` phù hợp với invariant của chính row như `available_quantity >= 0`; PostgreSQL lưu ý không nên dùng `CHECK` để enforce invariant phụ thuộc dữ liệu row khác/table khác, mà nên dùng các constraint phù hợp như `UNIQUE`, `FOREIGN KEY` hoặc cơ chế transactional khác. ⁶

Atomic inventory update:

```

UPDATE ticket_inventories
SET available_quantity = available_quantity - :quantity
WHERE ticket_category_id = :ticket_category_id

```

```
    AND available_quantity >= :quantity
RETURNING ticket_category_id, available_quantity;
```

Interpretation:

```
1 returned row → reservation succeeded
0 returned rows → insufficient inventory / no matching inventory
```

PostgreSQL `UPDATE` supports a conditional `WHERE` clause and `RETURNING`, nên application có thể phân biệt thành công/thất bại dựa trên row thực sự được update thay vì làm unsafe

`SELECT → check → UPDATE`. 8

Atomic voucher quota update:

```
UPDATE vouchers
SET used_count = used_count + 1
WHERE id = :voucher_id
    AND status = 'ACTIVE'
    AND starts_at <= NOW()
    AND expires_at > NOW()
    AND used_count < usage_limit
RETURNING id, used_count;
```

Voucher eligibility khác như applicable concert/category có thể validate trước hoặc kết hợp trong query; điểm cốt lõi là **quota mutation phải conditional tại DB**.

Transaction skeleton:

```
BEGIN;

-- reserve inventory atomically
-- reserve voucher quota atomically
-- create booking
-- create booking items
-- create voucher redemption

COMMIT;
```

Nếu booking gồm nhiều category, document rằng application xử lý category IDs theo deterministic order để giảm nguy cơ deadlock; PostgreSQL documentation cũng chỉ ra ordering các rows trong batch update có thể giúp tránh deadlock khi concurrent commands dùng cùng ordering. 8

Expiry worker plus point:

```
SELECT id
FROM bookings
```

```
WHERE status = 'RESERVED'
  AND expires_at <= NOW()
ORDER BY expires_at, id
LIMIT 100
FOR UPDATE SKIP LOCKED;
```

`SKIP LOCKED` không thích hợp cho general-purpose consistent reads, nhưng PostgreSQL chính thức mô tả nó là phù hợp để giảm lock contention khi nhiều consumers truy cập một queue-like table; đó chính xác là behavior mong muốn của nhiều expiry workers cạnh tranh lấy batch work. ⁹

Index rationale table:

Query/workload	Index
Expired reservations	<code>(status, expires_at)</code>
User booking history	<code>(user_id, created_at DESC)</code>
Operation filter by concert/status	<code>(concert_id, status, created_at DESC)</code>
Voucher lookup	<code>UNIQUE(code)</code>
Idempotency lookup	<code>UNIQUE(user_id, idempotency_key)</code>
Voucher per-user rule	<code>UNIQUE(voucher_id, user_id)</code>

Đừng chỉ liệt kê indexes. Với mỗi index, chỉ ra **query pattern** nào cần nó.

Reviewer quick-check — DB Design

- ☐ ERD khớp migration thực tế.
- ☐ Mọi critical invariant có enforcement tương ứng.
- ☐ Atomic inventory SQL nhìn thấy trực tiếp.
- ☐ Voucher quota không dùng read-then-write.
- ☐ Idempotency có DB uniqueness.
- ☐ Price được snapshot.
- ☐ Expiry query có index.
- ☐ Transaction boundary được mô tả.
- ☐ Không dùng `CHECK` sai cho cross-table invariant.
- ☐ Có reasoning về deadlock/contention.

ASSUMPTIONS_AND_LIMITATIONS.md

Đây là **explicit deliverable**, không phải disclaimer phụ. Assignment thậm chí đưa ví dụ rằng bạn có thể tự giới hạn booking states hoặc seed vouchers thay vì làm full create/update/delete API, miễn là giải thích scope. `filecite\turnOfffile0`

Template:

```
# Assumptions and Limitations
```


Business Assumptions

ID	Assumption	Reason	Impact
A-01	Tickets are capacity-based, not seat-based	Scope control	No seat map
A-02	Reservation TTL is 10 minutes	Avoid indefinite stock hold	Expiry worker required
A-03	One voucher redemption per user	Anti-abuse rule	UNIQUE(voucher_id,user_id)
A-04	Payment gateway is out of scope	Assessment scope	Operator/simulated confirmation
A-05	Historical booking price is immutable	Accounting integrity	Price snapshot

Implemented

[feature matrix]

Partially Implemented

[feature matrix]

Not Implemented

[feature matrix]

Known Limitations

[technical limitations]

Production Improvements

[future architecture]

Security Assumptions

[auth simplifications, seeded credentials, secrets handling]

Feature matrix:

Capability	Status	Notes/API
Concert browse	Done	GET /concerts
Ticket availability	Done	Informational snapshot
Ticket reserve	Done	POST /bookings
Retry idempotency	Done	Idempotency-Key
Voucher application	Done	Atomic quota

Capability	Status	Notes/API
Payment gateway	Out of scope	No external integration
Seat assignment	Out of scope	Category-capacity model
Refund	Out of scope	—

Reviewer quick-check

- [] Assumptions có ID.
- [] Mỗi assumption chỉ ra impact.
- [] Done/not done rõ ràng.
- [] Không cố che feature thiếu.
- [] Không mô tả future feature như đã implement.
- [] Scope khớp API và tests.

ADRs

ADRs không được PDF gọi tên, nhưng tôi sẽ **treat as mandatory-for-quality** vì reviewer muốn hiểu trade-offs. Microsoft Well-Architected guidance hiện khuyến nghị ADR ghi problem/context, options considered, decision outcome, trade-offs và status; ADR nên ngắn, factual và có rationale chứ không biến thành một design guide khác. ¹⁰

Chỉ cần 3–5 ADR chất lượng:

```
001-use-modular-monolith.md
002-use-postgresql-as-consistency-boundary.md
003-use-conditional-update-for-inventory.md
004-use-idempotency-key-and-request-fingerprint.md
005-use-db-backed-expiry-worker.md
```

Template:

ADR-003: Use Conditional Database Update for Ticket Reservation

Status

Accepted

Context

Ticket inventory is finite and concurrent booking attempts can target the same ticket category.

A SELECT-then-UPDATE implementation permits a race where multiple transactions observe the same available quantity.

Decision Drivers

- no overselling
- simple failure semantics
- stated load does not require distributed locking
- PostgreSQL is the inventory source of truth

Options Considered

Conditional UPDATE

Pros:

- atomic at database boundary
- simple
- no secondary lock service

Cons:

- hot row contention remains possible

SELECT ... FOR UPDATE

Pros:

- explicit locking and easy reasoning

Cons:

- waiting transactions and longer lock holding if transaction grows

Redis Distributed Lock

Pros:

- external coordination

Cons:

- adds a second failure/consistency boundary

Decision

Use conditional UPDATE with `available_quantity >= requested_quantity`.

Consequences

Positive:

- inventory cannot be decremented below requested capacity by racing updates
- simple handling based on affected rows

Negative:

- a highly contended category becomes a hot database row
- transaction duration must remain short

Verification

CONC-001: 100 concurrent requests against `inventory=10`.

Expected exactly 10 reservations and final `available_quantity=0`.

Reviewer quick-check

- [] Có alternatives.
- [] Có negative consequences.
- [] Decision trở tới test/evidence.
- [] ADR ngắn hơn System Design.
- [] ADR không viết sau để hợp thức hóa buzzword.

API, codebase và reproducibility artifacts

README.md

README nên được viết cho reviewer, không phải cho chính tác giả.

Mục tiêu thực tế: reviewer clone repo và không cần hỏi bạn “chạy kiểu gì?”.

Template:

```
# Concert Ticket Booking Backend

## What This Is

[2-4 sentences]

## Architecture at a Glance

[small architecture diagram]

## Key Guarantees

- concurrency-safe ticket reservation
- retry-safe booking creation
- bounded voucher usage
- reservation expiration

## Prerequisites

- Docker / Docker Compose
- optional local runtime version if running without Docker

## Quick Start

cp .env.example .env
docker compose up --build -d

## Database Migration

[command]

## Seed Data
```

[command]

Verify

curl http://localhost:3000/health

API Documentation

Swagger:

http://localhost:3000/docs

OpenAPI:

openapi/openapi.yaml

Test Accounts / Seed Data

Type	Value
Customer	...
Operator	...
Concert	...
VIP inventory	10
Voucher	FLASH20

Example Booking

[curl]

Run Tests

[unit command]

[integration command]

[concurrency command]

[k6 command]

Documentation

- Executive Summary
- System Design
- Database Design
- Assumptions / Limitations
- ADRs
- Performance Report

Scope

[compact link/summary]

Repository Structure

[tree]

Docker Compose là công cụ phù hợp cho “clone-and-run” vì Compose định nghĩa services/networks/volumes trong một configuration và khởi động chúng thống nhất. ¹¹

Coding guideline

Đề gọi tên trực tiếp coding guideline/convention, bao gồm cách code một API mới và cách chạy unit test. `filecite\turn0file0`

Không cần viết coding bible. Khoảng 2–4 trang là đủ:

Coding Guidelines

Module Boundaries

Controller → Application/Use Case → Repository/Domain

Controllers:

- HTTP parsing
- validation handoff
- response mapping

Use cases:

- business workflow
- transactions
- invariants

Repositories:

- persistence operations

Adding a New API

1. Define request/response contract.
2. Add validation DTO/schema.
3. Add application use case.
4. Add repository operations when needed.
5. Define business/error behavior.
6. Add OpenAPI docs.
7. Add unit/integration tests.
8. Add Postman example when externally exposed.

Transaction Rules

- Do not call external network services inside critical DB transactions.
- Inventory/voucher booking mutations belong to the booking transaction.
- Keep transactions short.
- Use deterministic resource ordering for multi-category reservation.

Error Convention

[standard envelope/problem details]

Naming

[project-specific rules]

Testing Convention

[test file naming]

[commands]

[arrange-act-assert convention]

Logging

[fields permitted / prohibited]

Migration + seed

Đề yêu cầu local setup và Postman collection phải chạy được với local setup; migration/seed là cách biến requirement đó thành reproducible artifact thay vì buộc reviewer manually create DB data. Đây là recommendation suy ra trực tiếp từ submission requirement. [filecite](#) [turn0file0](#)

Migration acceptance checklist:

Requirement	Expected
Fresh DB	Chạy toàn bộ migration từ zero
Repeatability	Tool biết migration nào đã apply
Constraints	Khớp DB design
Indexes	Khớp documented indexes
Seed separation	Production schema migration không lẫn arbitrary demo data
Reviewer data	Có seed deterministic
Failure	Command fail với non-zero status

Seed tối thiểu:

Customer user

Operator user

Published concert

└─ VIP qty=10

└─ Standard qty=100

Voucher FLASH20

```
|— active
|— global limit=3
|— per-user limit=1
```

Reviewer sẽ có ngay dữ liệu để chạy edge-case tests.

OpenAPI / Swagger

OpenAPI specification cho phép mô tả HTTP API theo machine-readable format gồm paths, schemas, parameters/headers, responses và examples; đó là lý do nên coi `openapi.yaml` là API source artifact thay vì chỉ screenshot Swagger UI. ¹

`POST /bookings` phải document đầy đủ hơn endpoint CRUD thông thường:

```
/bookings:
  post:
    summary: Reserve tickets and create a booking
    operationId: createBooking

    parameters:
      - in: header
        name: Idempotency-Key
        required: true
        schema:
          type: string
          minLength: 16
          maxLength: 128
        description: >
          Unique key for one logical booking attempt.
          Reusing the key with a different request payload is rejected.

    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/CreateBookingRequest'
          example:
            concertId: "2e..."
            items:
              - ticketCategoryId: "vip-id"
                quantity: 2
            voucherCode: FLASH20

    responses:
      '201':
        description: Booking created
      '409':
        description: Booking conflict or request currently in progress
```



```
'422':
  description: Business validation or idempotency payload conflict
```

Request example:

```
POST /bookings HTTP/1.1
Content-Type: application/json
Idempotency-Key: 7c48b7c3-3611-4ff6-9ddb-fc66c9b9c31e
```

```
{
  "concertId": "concert_01",
  "items": [
    {
      "ticketCategoryId": "vip_01",
      "quantity": 2
    }
  ],
  "voucherCode": "FLASH20"
}
```

Response:

```
{
  "id": "booking_01",
  "status": "RESERVED",
  "items": [
    {
      "ticketCategoryId": "vip_01",
      "quantity": 2,
      "unitPrice": 2000000
    }
  ],
  "subtotal": 4000000,
  "discountAmount": 800000,
  "finalAmount": 3200000,
  "expiresAt": "2026-08-23T10:10:00Z"
}
```

Một nuance đáng ghi trong API docs: IETF từng phát triển `Idempotency-Key` draft cho POST/PATCH, trong đó đề xuất key không được reuse cho payload khác, có thể dùng fingerprint, completed retry trả lại kết quả cũ, concurrent duplicate có thể conflict. Tuy nhiên revision `-07` hiện **đã hết hạn từ tháng 4/2026 và chưa trở thành RFC**, nên hãy cite nó trong design như một industry-design reference, không trình bày như tiêu chuẩn HTTP chính thức. ¹²

Request fingerprint design:

```
fingerprint =
SHA256(
  authenticated_user_id
  + normalized_concert_id
  + canonical_sorted_items
  + normalized_voucher_code
)
```

Behavior table:

Situation	Response
New key	Process booking
Same key + same fingerprint + completed	Replay stored result
Same key + different fingerprint	Reject
Same key + request still processing	Conflict/retry policy
Missing key	Reject if endpoint contract requires it

Database:

```
CREATE UNIQUE INDEX uq_idempotency_user_key
ON idempotency_records(user_id, idempotency_key);
```

Đừng dùng memory cache làm correctness boundary cho idempotency.

Error contract

Một plus point sạch là dùng RFC 9457 Problem Details thay vì tự nghĩ error envelope khó tiêu chuẩn hóa. RFC 9457 là IETF Standards Track, định nghĩa `application/problem+json` cho machine-readable HTTP API error details và thay thế RFC 7807. ¹³

Ví dụ:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json
```

```
{
  "type": "urn:ticket-booking:problem:inventory-exhausted",
  "title": "Ticket inventory is no longer available",
  "status": 409,
  "detail": "The requested VIP quantity cannot be reserved.",
  "instance": "/bookings/requests/req_123",
  "code": "INSUFFICIENT_TICKET_INVENTORY",
```

```
"traceId": "c677..."
}
```

OpenAPI quick-check:

- [] Every public operation documented.
- [] `Idempotency-Key` visible on booking endpoint.
- [] Request examples exist.
- [] Success response example exists.
- [] Business errors documented.
- [] Authentication/role behavior documented.
- [] `expiresAt`, price and booking status semantics explained.
- [] Availability endpoint warns that displayed availability is a snapshot.

Postman

Postman automatically generates collection documentation containing method, auth, URL, headers, request/response structures and examples; vì vậy collection nên được coi là **executable walkthrough**, không phải chỉ danh sách requests. ¹⁴

Folder layout:

```
Ticket Booking
|
|— Health
|— Customer
|   |— Browse concerts
|   |— Ticket availability
|   |— Create booking
|   |— Retry same booking
|   |— Get booking
|
|— Operation
|   |— List bookings
|   |— Booking detail
|   |— Change booking status
|
|— Edge Cases
|   |— Sold out
|   |— Expired voucher
|   |— Exhausted voucher
|   |— Same idempotency key / different payload
|   |— Invalid booking transition
```

Environment:

```
{
  "baseUrl": "http://localhost:3000",
  "customerToken": "...",
}
```

```

    "operatorToken": "...",
    "concertId": "...",
    "vipCategoryId": "...",
    "voucherCode": "FLASH20"
  }

```

Mỗi core request nên có Postman tests tối thiểu:

```

pm.test("status is expected", () => {
  pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});

pm.test("booking id exists", () => {
  const json = pm.response.json();
  pm.expect(json.id).to.exist;
});

```

Reviewer quick-check — API artifacts

Check	Pass
Swagger	UI mở trên local
OpenAPI	Source file committed
Postman	Collection committed
Environment	Local environment committed, không chứa secret thật
Seed IDs	Requests tự lấy hoặc documented
Idempotency	Có retry example
Errors	Có sold-out/voucher/transition examples
Cross-reference	README chỉ thẳng tới Swagger/Postman

Testing, concurrency, load evidence và CI

Đây là phần có khả năng tạo khác biệt lớn nhất giữa một submission “đúng ý tưởng” và một submission **có bằng chứng**.

TEST_STRATEGY.md

Không nên report test chỉ bằng “87 tests passed”. Hãy map tests về invariants.

Test taxonomy:

Layer	Dùng để chứng minh
Unit	Pure business policy/state calculations

Layer	Dùng để chứng minh
Integration	SQL/transaction/constraints thật
API/E2E	HTTP contract + auth + error mapping
Concurrency	Race conditions và finite-resource guarantees
Load	Behavior dưới flash-sale traffic

Unit test set:

ID	Case	Assertion
UNIT-001	Calculate subtotal	Correct amount
UNIT-002	Apply valid percentage voucher	Correct discount/final
UNIT-003	Expired voucher	Rejected
UNIT-004	Invalid booking transition	Rejected
UNIT-005	RESERVED → CONFIRMED	Allowed
UNIT-006	EXPIRED → CONFIRMED	Rejected
UNIT-007	Duplicate category normalization	Deterministic behavior

Integration tests:

ID	Scenario	Expected assertions
INT-001	Successful booking	booking created; inventory decreased
INT-002	Insufficient stock	no booking; inventory unchanged
INT-003	Voucher succeeds	redemption created; quota incremented
INT-004	Stock reserved then voucher fails	entire transaction rolls back
INT-005	Reservation expiry	EXPIRED + stock returned
INT-006	Same voucher/user twice	second redemption rejected
INT-007	Ticket price later changes	old booking price unchanged
INT-008	Invalid operator transition	booking state unchanged

PostgreSQL's isolation model matters here: at default Read Committed level nonrepeatable/phantom behavior remains possible, nên correctness không nên dựa vào "mình vừa SELECT thấy nó như vậy"; write-side conditions, locks và constraints phải được test thật. ¹⁵

Concurrency test suite

Đây nên được xem là **primary WOW artifact**.

CONC-001 — overselling

```
Initial inventory = 10
Concurrent requests = 100
Requested quantity per request = 1
```

Assertions:

```
successful bookings      = 10
sold-out responses       = 90
final available quantity = 0
inventory < 0 ever       = false
booking items sold       = 10
```

Test code phải kiểm tra state cuối trong DB, không chỉ count HTTP 201.

CONC-002 — duplicate retries

```
20 concurrent requests
same authenticated user
same Idempotency-Key
same payload
```

Expected:

```
logical bookings created = 1
inventory decrement      = once
unique booking IDs      = 1
voucher consumption      = once
```

CONC-003 — voucher race

```
voucher remaining quota = 1
20 concurrent distinct users
```

Expected:

```
successful redemption = 1
quota exceeded        = false
used_count            = limit
```

CONC-004 — confirm vs expire race

```
booking close to expires_at
```

```
worker:
RESERVED → EXPIRED

concurrent API:
RESERVED → CONFIRMED
```

Assertion:

```
exactly one legal outcome wins

either:
CONFIRMED and inventory remains consumed

or:
EXPIRED and inventory is released

never:
CONFIRMED + inventory released
never:
EXPIRED + inventory consumed without documented reason
```

Conditional status writes are useful:

```
UPDATE bookings
SET status = 'CONFIRMED'
WHERE id = :booking_id
  AND status = 'RESERVED'
  AND expires_at > NOW()
RETURNING id;
```

PERFORMANCE_REPORT.md

Nó phải chứa **environment**, nếu không latency numbers gần như không có giá trị so sánh.

Template:

```
# Performance and Concurrency Report

## Test Metadata

| Item | Value |
|---|---|
| Commit SHA | ... |
| Timestamp | ... |
| Machine | ... |
| CPU / RAM | ... |
| Runtime version | ... |
```

```

| PostgreSQL version | ... |
| Containerized | yes/no |

## Scenario

Initial inventory: 10
Concurrent booking attempts: 100
Quantity/request: 1

## Correctness Results

| Metric | Expected | Actual | Pass |
|---|---:|---:|---|
| Successful bookings | 10 | ____ | ____ |
| Sold-out results | 90 | ____ | ____ |
| Final inventory | 0 | ____ | ____ |
| Duplicate booking | 0 | ____ | ____ |
| Negative inventory | 0 | ____ | ____ |

## Performance Results

| Metric | Result |
|---|---:|
| Requests | ____ |
| Throughput | ____ req/s |
| p50 | ____ ms |
| p95 | ____ ms |
| p99 | ____ ms |
| Unexpected 5xx | ____ |
| Expected business rejections | ____ |

## Interpretation

[What bottleneck appeared?]

## Known Limitations

[Local-machine benchmark caveats]

```

Quan trọng: **business rejection như SOLD_OUT không nên cộng chung với unexpected HTTP/system failures**. Một flash sale mà 90 khách nhận controlled sold-out response có thể vẫn hoàn toàn healthy.

k6 flash-sale script

k6 scenarios cho phép mô hình hóa workload patterns bằng virtual users/executors; thresholds là pass/fail criteria, và khi threshold fail thì k6 có thể trả non-zero exit status, rất phù hợp để đưa performance regression vào CI hoặc ít nhất làm reproducible artifact. ¹⁶

Ví dụ:


```

import http from 'k6/http';
import { check } from 'k6';

export const options = {
  scenarios: {
    flash_sale: {
      executor: 'constant-arrival-rate',
      rate: 10,
      timeUnit: '1s',
      duration: '60s',
      preAllocatedVUs: 20,
      maxVUs: 100,
    },
  },

  thresholds: {
    http_req_duration: ['p(95)<500', 'p(99)<1000'],
    http_req_failed: ['rate<0.01'],
    checks: ['rate>0.99'],
  },
};

export default function () {
  const payload = JSON.stringify({
    concertId: __ENV.CONCERT_ID,
    items: [
      {
        ticketCategoryId: __ENV.CATEGORY_ID,
        quantity: 1,
      },
    ],
  });

  const params = {
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${__ENV.TOKEN}`,
      'Idempotency-Key': `${__VU}-${__ITER}-${Date.now()}`,
    },
  };

  const res = http.post(`${__ENV.BASE_URL}/bookings`, payload, params);

  check(res, {
    'controlled booking response': (r) =>
      [201, 409, 422].includes(r.status),
  });
}

```

Threshold values như `p95<500` ở trên **chỉ là template**, không phải SLO do đề bài đặt ra. Hãy chọn target và giải thích rationale hoặc chỉ report observed result nếu không muốn tự đặt production SLO.

k6 có thể export granular JSON và custom end-of-test summary; vì vậy nên commit summary/result artifact thay vì screenshot terminal duy nhất. ¹⁷

Commands:

```
k6 run \
  --out json=test-results/load/raw.json \
  tests/load/flash-sale.k6.js
```

hoặc custom `handleSummary()` để sinh:

```
test-results/load/summary.json
test-results/load/summary.html
```

k6 chính thức report các percentile như p90/p95/p99 và hỗ trợ custom summary output. ¹⁸

Charts nên đưa vào report:

Chart	Lý do
Request latency p50/p95/p99	Nhìn tail latency
Throughput over time	Thấy degradation
Business success vs sold-out	Correctness/business behavior
Unexpected errors over time	Stability
Optional DB lock wait	Contention insight

Không cần Grafana dashboard nếu chỉ có 48 giờ. Một table + 2 chart từ reproducible results đã đủ.

Docker + CI

Suggested `compose.yaml`:

```
services:
  db:
    image: postgres:18
    environment:
      POSTGRES_DB: ticket_booking
      POSTGRES_USER: ticket
      POSTGRES_PASSWORD: ticket
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ticket -d ticket_booking"]
      interval: 5s
```

```

        timeout: 3s
        retries: 10

    api:
        build: .
        env_file:
            - .env
        depends_on:
            db:
                condition: service_healthy
        ports:
            - "3000:3000"

```

Docker documentation khuyến nghị Compose cho việc định nghĩa services/networks/volumes và sử dụng trong development/testing; health/reproducibility của DB dependency là một giá trị lớn cho reviewer. ¹¹

Suggested quality gate:

```

install
↓
lint
↓
typecheck
↓
unit tests
↓
start PostgreSQL
↓
migrations
↓
integration tests
↓
build

```

GitHub Actions hỗ trợ PostgreSQL service containers trực tiếp trong workflow và tài liệu chính thức cung cấp health-check patterns cho database-dependent tests. ¹⁹

Ví dụ:

```

name: CI

on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

```

```

services:
  postgres:
    image: postgres:18
    env:
      POSTGRES_USER: ticket
      POSTGRES_PASSWORD: ticket
      POSTGRES_DB: ticket_booking_test
    ports:
      - 5432:5432
    options: >-
      --health-cmd pg_isready
      --health-interval 5s
      --health-timeout 5s
      --health-retries 10

steps:
  - uses: actions/checkout@v6

  - name: Install dependencies
    run: <install-command>

  - name: Lint
    run: <lint-command>

  - name: Unit tests
    run: <unit-test-command>

  - name: Migrate test DB
    run: <migration-command>

  - name: Integration tests
    run: <integration-test-command>

  - name: Build
    run: <build-command>

```

GitHub's own build/test guidance recommends using the same build and test commands in CI that are used locally; đây là lý do README và workflow nên gọi chung scripts thay vì duplicate logic khác nhau.

20

Reviewer quick-check — testing

- [] Tests map về invariants.
- [] Integration test dùng database thật, không mock persistence cho transaction tests.
- [] Có rollback test.
- [] Có overselling concurrency test.
- [] Có idempotency concurrency test.
- [] Có voucher concurrency test.
- [] Có expiry/confirm race test.
- [] Report chứa environment + commit SHA.

- [] Không có benchmark numbers fabricated.
- [] Business rejection và system errors được tách riêng.
- [] Có single command hoặc documented commands để reproduce.

Các artifact WOW nên ưu tiên

Không nên cố implement toàn bộ phần này. Với 48 giờ, thứ tự return-on-effort tôi đánh giá là:

Extra	Giá trị reviewer	Cost/risk
Concurrency tests + report	Rất cao	Thấp-vừa
Strong idempotency + fingerprint	Rất cao	Vừa
Expiry worker + SKIP LOCKED	Cao	Vừa
Audit history	Cao	Thấp
Structured logs + metrics	Cao	Thấp
CI	Cao	Thấp
Transactional outbox	Khá cao	Vừa
Redis	Trung bình	Vừa
Kafka	Thấp cho scope này	Cao
Kubernetes	Rất thấp cho assessment	Cao

Transactional outbox design

Chỉ thêm nếu core booking tests đã hoàn chỉnh.

Problem:

```

DB transaction commits
    ↓
application publishes BookingConfirmed
    ↓
process crashes before publish

booking exists
event disappears

```

AWS Prescriptive Guidance mô tả transactional outbox chính xác để xử lý dual-write problem này: domain data và outbox event được ghi trong cùng transaction; publisher riêng đọc committed outbox và gửi ra broker/downstream system. AWS cũng lưu ý delivery có thể tạo duplicates, nên consumers vẫn phải idempotent. ²¹

Schema:

```

CREATE TABLE outbox_events (
  id uuid PRIMARY KEY,
  aggregate_type text NOT NULL,
  aggregate_id uuid NOT NULL,
  event_type text NOT NULL,
  payload jsonb NOT NULL,
  created_at timestamptz NOT NULL DEFAULT NOW(),
  published_at timestamptz NULL
);

CREATE INDEX idx_outbox_unpublished
ON outbox_events(created_at)
WHERE published_at IS NULL;

```

Transaction:

```

BEGIN

UPDATE bookings
SET status = 'CONFIRMED'
...

INSERT INTO outbox_events(
  event_type,
  aggregate_id,
  payload
)
VALUES (
  'BookingConfirmed',
  ...,
  ...
);

COMMIT

```

Outbox design document nên ghi rõ:

```

Implemented:
Postgres outbox table + polling worker

Not implemented:
Kafka / RabbitMQ

Future:
replace publisher destination without changing booking transaction

```

Không nên ghi “exactly once messaging” trừ khi bạn thực sự chứng minh semantics đó. AWS guidance cũng cảnh báo duplicate messages và yêu cầu consumer idempotency trong at-least-once style delivery.

21

Observability artifact

OpenTelemetry hiện tổ chức telemetry thành các signals như traces, metrics và logs; bạn không nhất thiết phải deploy full OTel stack, nhưng dùng cùng mental model trong `OBSERVABILITY.md` là đủ để cho thấy operational thinking. 22

Structured log example:

```
{
  "timestamp": "2026-08-23T10:00:00Z",
  "level": "INFO",
  "event": "booking.created",
  "traceId": "84b...",
  "requestId": "req_123",
  "bookingId": "booking_123",
  "concertId": "concert_1",
  "userId": "user_4",
  "durationMs": 42
}
```

Không log:

```
password
JWT token
payment credential
full sensitive payload
```

Metrics catalog:

Metric	Type	Ý nghĩa
<code>booking_attempt_total</code>	Counter	All attempts
<code>booking_created_total</code>	Counter	Successful reservations
<code>booking_rejected_total{reason}</code>	Counter	Sold out/voucher/etc.
<code>booking_duration_ms</code>	Histogram	Booking latency
<code>inventory_reservation_conflict_total</code>	Counter	Contention/business conflict
<code>idempotency_replay_total</code>	Counter	Retry handling
<code>voucher_redemption_total{result}</code>	Counter	Voucher behavior
<code>reservation_expired_total</code>	Counter	TTL behavior

Metric	Type	Ý nghĩa
expiry_worker_batch_size	Histogram	Worker processing
outbox_pending	Gauge	Optional async backlog

OBSERVABILITY.md nên trả lời:

How do we detect booking failures?
 How do we distinguish sold-out from system failure?
 How do we correlate one HTTP request with booking logs?
 How would Operations inspect suspicious manual changes?
 What metrics would alert us to flash-sale degradation?

Audit log

Operation team theo dõi để có thể xử lý failed/suspicious bookings và manual update booking status, vì vậy audit history là extra rất sát business requirement chứ không phải generic compliance decoration.

filecitefileturn0file0

Schema:

```
CREATE TABLE booking_status_history (
  id uuid PRIMARY KEY,
  booking_id uuid NOT NULL REFERENCES bookings(id),
  from_status text NOT NULL,
  to_status text NOT NULL,
  changed_by uuid,
  change_source text NOT NULL,
  reason text,
  created_at timestamptz NOT NULL DEFAULT NOW()
);
```

Sample:

```
{
  "bookingId": "BK-001",
  "fromStatus": "RESERVED",
  "toStatus": "CANCELLED",
  "changedBy": "operator-07",
  "changeSource": "OPERATION_DASHBOARD",
  "reason": "Suspected duplicate reservation"
}
```

Reviewer quick-check:

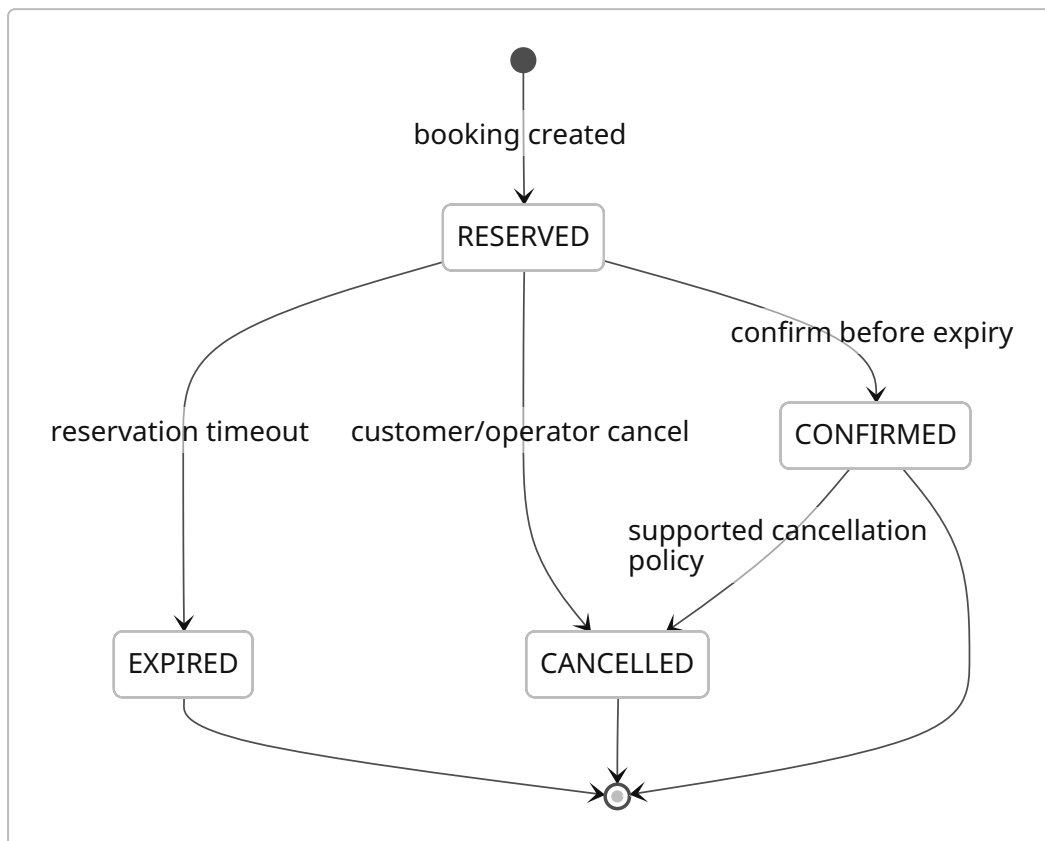
- [] Manual state changes lưu actor.
- [] Lưu previous/new state.

- [] Có timestamp.
- [] Có optional/required reason policy.
- [] Audit row không thay thế booking state machine; cả hai cùng tồn tại.

Visual asset pack và các template copy-paste

Visual documentation nên được commit dưới dạng Mermaid source bên cạnh rendered preview nếu cần. Mermaid chính thức hỗ trợ ER diagrams, interaction/sequence diagrams và state transitions; state diagram đặc biệt phù hợp với booking finite-state model. ²³

Booking state machine

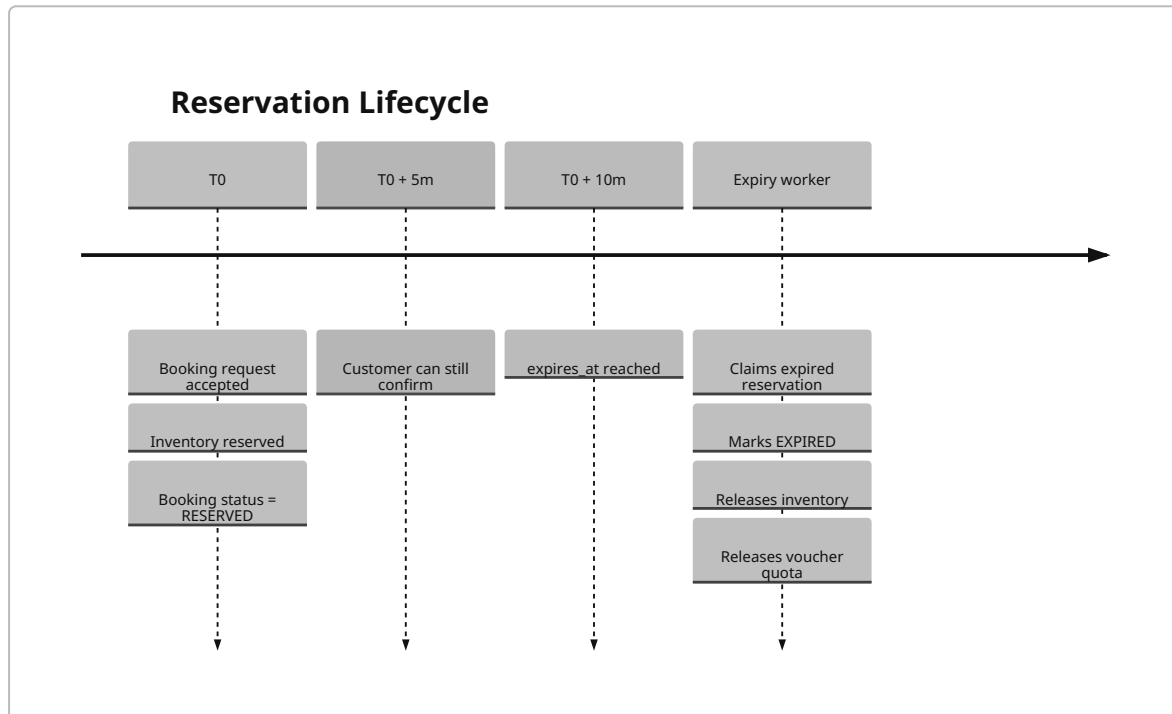


Kèm transition matrix, vì diagram không thay thế specification:

Current	Command	Target	Allowed?	Resource effect
RESERVED	Confirm	CONFIRMED	Yes	Keep inventory consumed
RESERVED	Expire	EXPIRED	Yes	Release inventory/voucher
RESERVED	Cancel	CANCELLED	Yes	Release inventory/voucher
EXPIRED	Confirm	CONFIRMED	No	None
CANCELLED	Confirm	CONFIRMED	No	None
CONFIRMED	Cancel	CANCELLED	Depends scope	Document explicitly

Mermaid state diagrams mô hình hóa states và transitions trực tiếp, gồm start/end nodes và transition labels; đó là representation rất tự nhiên cho booking lifecycle. ²⁴

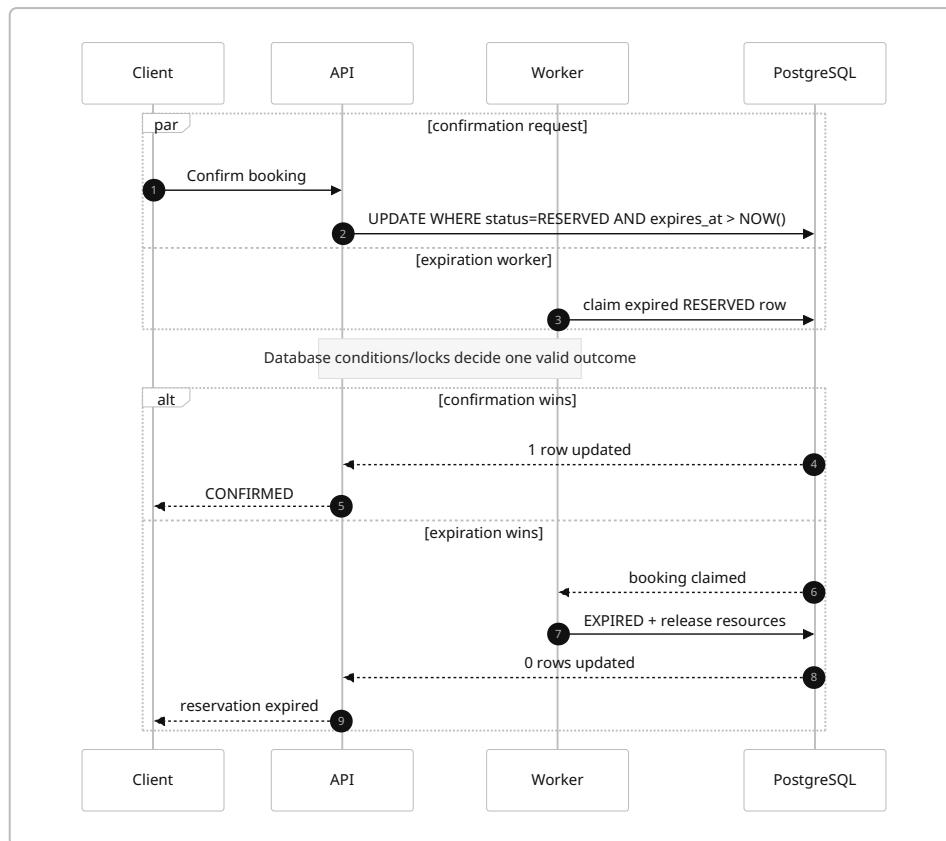
Reservation lifecycle timeline



Mermaid hiện mô tả timeline syntax chính thức nhưng đánh dấu loại diagram này là experimental; vì vậy commit sequence/state diagrams như canonical source, còn timeline này chỉ phục vụ communication. ²⁵

Expire-vs-confirm race

Một sequence diagram tốt hơn timeline để giải thích race:



Reviewer evidence table template

Mỗi performance/concurrency result nên dùng cùng format:

Test ID	Initial State	Concurrency	Expected	Actual	Result
CONC-001	Inventory=10	100	10 success, stock=0	—	PASS/ FAIL
CONC-002	New idem key	20	one logical booking	—	PASS/ FAIL
CONC-003	Voucher remaining=1	20	one redemption	—	PASS/ FAIL
CONC-004	RESERVED near expiry	2 racers	one consistent terminal outcome	—	PASS/ FAIL

SQL evidence box

Đặt một box kiểu này trong Database Design:

```

## Critical Database Guarantees

### Inventory never goes negative

```sql

```

```
UPDATE ticket_inventories
SET available_quantity = available_quantity - :qty
WHERE ticket_category_id = :category_id
 AND available_quantity >= :qty
RETURNING available_quantity;
```

Guarantee: The write succeeds only while sufficient inventory exists.

Evidence: CONC-001.

## Duplicate logical requests

```
UNIQUE(user_id, idempotency_key)
```

Guarantee: Two transactions cannot persist two idempotency records with the same scope/key.

Evidence: CONC-002.

## Voucher global quota

```
UPDATE vouchers
SET used_count = used_count + 1
WHERE id = :voucher_id
 AND used_count < usage_limit
RETURNING used_count;
```

Evidence: CONC-003.

PostgreSQL unique constraints formally guarantee uniqueness over the constrained column set, và conditional updates/`RETURNING` cho phép chính database write xác định operation có thành công hay không. <sup>26</sup>

### Final submission checklist

Tôi sẽ dùng checklist này ngay trước khi gửi link:

Area	Final check
**Submission**	Tất cả artifacts nằm trong một folder/repo như đề yêu cầu.                                  

```

| **Assumptions** | Scope, done/not done, limitations explicit |
| **ADR** | Có context, alternatives, decision, consequences |
| **Migration** | Fresh database setup thành công |
| **Seed** | Có concert/category/inventory/voucher/users deterministic |
| **Docker** | `docker compose up --build` hoặc equivalent hoạt động |
| **Health** | Reviewer biết cách verify API/DB up |
| **Swagger** | Mở được local |
| **OpenAPI** | Source specification committed |
| **Postman** | Collection + local environment committed và testable |
| **Booking contract** | `Idempotency-Key`, request, responses, business
errors documented |
| **Errors** | Stable machine-readable error codes; ideally Problem Details |
| **Unit tests** | Pure business policies covered |
| **Integration tests** | DB transaction/rollback/constraints covered |
| **Concurrency** | Oversell/idempotency/voucher races tested |
| **Expiry** | Expiration and confirm-vs-expire race tested |
| **Performance** | Test environment + commit SHA + p95/p99 + failures
documented |
| **k6** | Script committed; result artifact reproducible |
| **Audit** | Optional manual-operation history demonstrated |
| **Observability** | Optional log/metric examples included |
| **Outbox** | Optional; clearly labeled implemented vs future |
| **CI** | Same core test/build commands as local |
| **Secrets** | Không commit production credential/token |
| **Consistency** | README, diagrams, migration, OpenAPI và code không
contradict nhau |
| **Demo** | Có một happy-path và ít nhất ba edge-case scenarios ready to run
|
| **Evidence** | Không có con số benchmark nào không thể reproduce |

```

### Bộ visual assets nên xuất hiện trong submission

Tối thiểu nên cung cấp đủ năm source diagrams sau:

```

```text
docs/diagrams/
├── architecture.mmd
│   └── modules + PostgreSQL + expiry worker
├── erd.mmd
│   └── booking/inventory/voucher/idempotency/audit
├── booking-sequence.mmd
│   └── POST /bookings transaction
├── booking-state.mmd
│   └── RESERVED / CONFIRMED / EXPIRED / CANCELLED
└── reservation-timeline.mmd
    └── reservation TTL / expiry lifecycle

```

Tôi sẽ xem **architecture + ERD + booking sequence + state machine** là mandatory visual documentation; timeline là optional communication aid. Mermaid hỗ trợ ba loại canonical diagrams đầu tiên trực tiếp, còn timeline hiện được chính Mermaid đánh dấu experimental. ²⁷

Tiêu chuẩn để biết submission đã thực sự “xong”

Một bộ submission tốt phải cho reviewer thực hiện chuỗi này mà không cần liên hệ tác giả:

```
Clone repository
  ↓
Read Executive Summary
  ↓
docker compose up
  ↓
run migration + seed
  ↓
open Swagger
  ↓
run Postman happy path
  ↓
retry same Idempotency-Key
  ↓
run sold-out scenario
  ↓
run unit/integration tests
  ↓
run concurrency test
  ↓
inspect result report
  ↓
read design + ADR rationale
```

Nếu flow trên chạy trơn tru, repo không chỉ đáp ứng các deliverables mà đề gọi tên; nó còn trực tiếp thể hiện điều assessment muốn đánh giá: **backend design reasoning, clean structure, explicit scope, và khả năng xử lý các workflow/race conditions quan trọng**. ^{filecite}^{turn0file0}

Với quỹ thời gian 48 giờ, điểm “wow” có ROI cao nhất không phải Kafka, Kubernetes hay một dashboard phức tạp. Nó là chuỗi bằng chứng:

```
Invariant
  ↓
Design decision
  ↓
Database guarantee
  ↓
Automated concurrent test
  ↓
Reproducible result report
```

Ví dụ hoàn chỉnh:

```
INV-01: inventory never negative
    ↓
ADR-003: conditional PostgreSQL update
    ↓
SQL WHERE available_quantity >= requested_quantity
    ↓
CONC-001: 100 racers for 10 tickets
    ↓
10 success / 90 controlled rejection / final inventory 0
```

Đó là dạng deliverable mà reviewer có thể **đọc, chạy, kiểm chứng và hỏi sâu trong interview**, đúng với tinh thần assessment hơn nhiều so với một repo có nhiều infrastructure nhưng thiếu proof of correctness. fileciteturn0file0

-
- 1 OpenAPI Specification v3.2.0
https://spec.openapis.org/oas/v3.2.0.html?utm_source=chatgpt.com
 - 2 7 23 Entity Relationship Diagrams | Mermaid
https://mermaid.js.org/syntax/entityRelationshipDiagram.html?utm_source=chatgpt.com
 - 3 11 Docker Compose | Docker Docs
https://docs.docker.com/compose?utm_source=chatgpt.com
 - 4 5 27 Sequence diagrams | Mermaid
https://mermaid.js.org/syntax/sequenceDiagram.html?utm_source=chatgpt.com
 - 6 26 PostgreSQL: Documentation: 18: 5.5. Constraints
<https://www.postgresql.org/docs/current/ddl-constraints.html>
 - 8 PostgreSQL: Documentation: 18: UPDATE
https://www.postgresql.org/docs/current/sql-update.html?utm_source=chatgpt.com
 - 9 PostgreSQL: Documentation: 18: SELECT
https://www.postgresql.org/docs/current/sql-select.html?utm_source=chatgpt.com
 - 10 Maintain an architecture decision record (ADR) - Microsoft Azure Well-Architected Framework | Microsoft Learn
https://learn.microsoft.com/en-us/azure/well-architected/architect-role/architecture-decision-record?utm_source=chatgpt.com
 - 12 The Idempotency-Key HTTP Header Field
https://www.ietf.org/archive/id/draft-ietf-httpapi-idempotency-key-header-07.html?utm_source=chatgpt.com
 - 13 RFC 9457: Problem Details for HTTP APIs
https://www.rfc-editor.org/rfc/rfc9457.html?utm_source=chatgpt.com
 - 14 Document a collection in Postman | Postman Docs
https://learning.postman.com/docs/publishing-your-api/document-a-collection?utm_source=chatgpt.com
 - 15 PostgreSQL: Documentation: 18: 13.2. Transaction Isolation
<https://www.postgresql.org/docs/current/transaction-iso.html>

16 Scenarios | Grafana k6 documentation

https://grafana.com/docs/k6/latest/using-k6/scenarios/?utm_source=chatgpt.com

17 JSON | Grafana k6 documentation

https://grafana.com/docs/k6/latest/results-output/real-time/json/?utm_source=chatgpt.com

18 Results output | Grafana k6 documentation

https://grafana.com/docs/k6/latest/get-started/results-output/?utm_source=chatgpt.com

19 Creating PostgreSQL service containers - GitHub Docs

https://docs.github.com/en/actions/tutorials/use-containerized-services/create-postgresql-service-containers?utm_source=chatgpt.com

20 Building and testing Node.js - GitHub Enterprise Server 3.17 Docs

https://docs.github.com/en/enterprise-server%403.17/actions/tutorials/build-and-test-code/nodejs?utm_source=chatgpt.com

21 Transactional outbox pattern - AWS Prescriptive Guidance

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html?utm_source=chatgpt.com

22 Documentation | OpenTelemetry

https://opentelemetry.io/docs/?utm_source=chatgpt.com

24 State diagrams | Mermaid

https://mermaid.js.org/syntax/stateDiagram?utm_source=chatgpt.com

25 Timeline Diagram | Mermaid

https://mermaid.js.org/syntax/timeline.html?utm_source=chatgpt.com